


Flash-WAM: Modality-Aware Distillation for World Action Models

Arman Akbari^{1*} Ci Zhang² Arash Akbari¹ Lin Zhao¹ Yixiao Chen¹
Weiwei Chen³ Xuan Zhang¹ Geng Yuan² Yanzhi Wang^{1,3}

¹Northeastern University ²University of Georgia ³EmbodimentX Inc.

 Project Page: flashwam.github.io

Abstract

World-action models (WAMs) jointly generate future video and robot actions through iterative diffusion, achieving strong performance on manipulation benchmarks but requiring tens of denoising steps, a cost that precludes real-time control. Step distillation has emerged as the natural remedy, but off-the-shelf methods break down in the joint video-action setting because video and action streams use different SNR-shifted noise schedules and reach training with substantially different marginal noise distributions, an asymmetry that single-modality distillation methods cannot accommodate. We introduce **Flash-WAM**, a modality-aware step-distillation framework inspired by consistency distillation that selects the consistency function for each modality to match its noise regime: a linear-gradient-scaling parametrization for the action stream’s low-noise regime, paired with a variance-preserving parametrization for the video stream’s high-noise regime, grounded in a structural analysis of the consistency-function family that characterizes the achievable gradient scaling under the consistency boundary condition. Instantiated on LingBot-VA, Flash-WAM compresses inference to a single step in each modality. On RoboTwin 2.0, this reduces per-chunk latency from 8.1 seconds to 348 ms on NVIDIA L40S, a $23\times$ speedup that enables real-time inference. Flash-WAM preserves task success on simulation benchmarks (85.5% RoboTwin 2.0, 95.7% LIBERO) and substantially recovers real-world performance (60% average on a Unitree G1 humanoid robot), while naive consistency distillation drops to 24% at the same step budget.

1 Introduction

Robotic foundation models aim to map perception and language to actions across diverse embodiments and tasks. The dominant approach has been Vision-Language-Action (VLA) policies [3, 11, 13, 40, 18, 25, 41], which adapt pretrained vision-language models (VLMs) to predict actions directly from observations. While effective in-distribution, VLA policies inherit a representation built for static visual understanding rather than physical dynamics, limiting their generalization to novel scenes, objects, and long-horizon tasks [26, 39]. This has motivated a shift toward using World Models for embodied AI [16, 10, 27], and in particular toward world-action models (WAMs) [15, 1, 35], which build on pretrained video generation backbones and jointly generate future visual states and the actions that produce them. By inheriting spatiotemporal priors from large-scale video pretraining, WAMs are emerging as the strongest candidate for general-purpose robotic foundation models.

*Correspondence to: akbari.ar@northeastern.edu

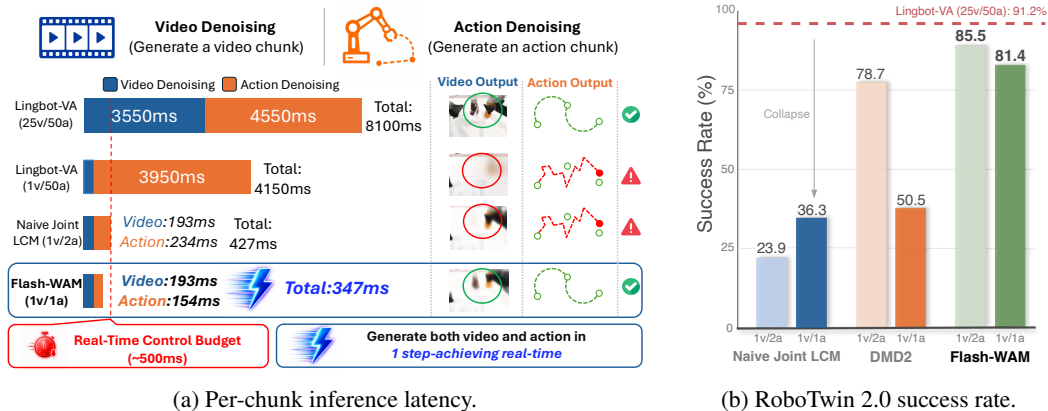


Figure 1: (a) Per-chunk inference latency on a single NVIDIA L40S. Flash-WAM brings WAM inference below the real-time control budget. (b) Average success rate on RoboTwin 2.0. Off-the-shelf distillation methods drop sharply, while Flash-WAM preserves *teacher*-level performance.

Most current WAMs realize joint video-action generation through a two-stage process: at each control step, the model first denoises a chunk of future video latents, then decodes the next action sequence conditioned on the predicted frames [17, 15, 1, 35]. Both stages are diffusion processes requiring iterative denoising, and both contribute substantially to per-chunk latency. As shown in Figure 1 (a), on the RoboTwin 2.0 [4] benchmark, the representative state-of-the-art WAM LingBot-VA [15] runs 25 video denoising steps and 50 action denoising steps per chunk, costing 3550 ms for video and 4550 ms for action on a single NVIDIA L40S GPU, for a total of 8.1 s per chunk (Figure 1). At this cost, real-time closed-loop control is out of reach. Existing WAMs [15, 35] mitigate this through engineering-level optimizations such as KV caching of past observations, partial denoising via noisy history augmentation, and asynchronous prediction-execution pipelines. These techniques reduce wall-clock latency without changing the underlying number of denoising steps, and remain orthogonal to methods that compress the denoising procedure itself.

A natural remedy is *step distillation*, which enables diffusion models to generate comparable results with far fewer denoising steps. Step distillation has been extensively developed for single-modality generation and has demonstrated substantial speedups for image and video synthesis [32, 22, 28, 36]. However, transferring these methods to the joint video-action setting is non-trivial: distribution-matching approaches such as DMD2 [36] require auxiliary score networks and **adversarial** training, which couple **awkwardly** with the **asymmetric** per-modality noise schedules used in WAMs; progressive distillation [28] requires a multi-stage training pipeline that scales poorly to the large pre-trained backbones that underlie modern WAMs. Consistency distillation [29, 22] is the most natural fit since it requires no auxiliary networks, integrates cleanly into existing flow-matching frameworks, and admits the analytical treatment we develop in this paper. Yet even this **otherwise reliable** approach does not carry over directly to WAMs. **Applying consistency distillation naively to a joint video-action model collapses task success rates from over 91% to as low as 23% on RoboTwin (Section 5.3),** posing a key obstacle to scaling WAMs toward real-time inference.

However, naively transferring consistency distillation to WAMs does not work. We uncover a fundamental incompatibility between standard consistency distillation and joint diffusion under asymmetric noise schedules. Video latents and action sequences have fundamentally different statistical properties: video is high-dimensional and structurally redundant, while actions are low-dimensional and precision-critical. To accommodate this asymmetry, WAMs employ different Signal-to-Noise-Ratio (SNR) shifted noise schedulers per modality [15, 35], matched to each modality’s information content. As a consequence, the two streams reach the consistency-distillation loss under substantially different marginal noise distributions: video noise concentrates at high σ , while action noise spreads across the full range with substantial mass at low σ . We show that existing consistency distillation methods (e.g., LCM [22]) provide gradient signal that vanishes *quadratically* as $\sigma \rightarrow 0$, leaving the action stream with negligible learning signal across most of its training distribution. Naive joint distillation therefore collapses action accuracy even when video reconstruction is preserved.

After identifying this incompatibility, we propose **Flash-WAM**, a step-distillation framework for joint video-action diffusion models. The core idea is to treat video and action distillation as fundamentally different problems with different gradient-signal requirements. Each modality receives a consistency function matched to where its training distribution concentrates: a variance-preserving choice for the high- σ regime where video trains, and a linear-gradient-scaling choice for the low- σ regime where actions train. We apply Flash-WAM to the released LingBot-VA model [15], the state-of-the-art open-source joint video-action diffusion model on manipulation benchmarks, whose parameter count is small enough to run on commodity edge hardware where step distillation has the most practical impact. Flash-WAM recovers task success with a single video step and one action step, achieving a $23\times$ speedup that brings per-chunk inference latency from 8.1 seconds to 348 ms on a single NVIDIA L40S, enabling real-time inference. Our contributions are as follows:

- **Diagnosis of joint-modality distillation failure.** We identify and characterize the structural failure mode that prevents **off-the-shelf** consistency distillation methods from succeeding in the joint video-action regime, with formal analysis and empirical experiments.
- **Modality-aware consistency distillation.** We propose Flash-WAM, a step-distillation framework that selects different members of the consistency-function family for each modality based on its noise regime. The framework is grounded in a structural analysis of the consistency-function family, characterizing the achievable gradient scaling under the consistency boundary condition.
- **Real-time WAM inference.** On LingBot-VA, Flash-WAM compresses inference to a single step in each modality on RoboTwin 2.0, reducing per-chunk latency from 8.1 seconds to 348 ms (up to $23\times$ speedup) on NVIDIA L40S. At one video step and two action steps, Flash-WAM recovers 85.5% on RoboTwin 2.0 and 95.7% on LIBERO; at one video step and one action step, it retains 81.4% and 95.1% respectively. On a Unitree G1 humanoid robot, Flash-WAM achieves 60% average success across three manipulation tasks, recovering most of the unaccelerated model’s 66.7% while video-only LCM collapses to 43.3%.

2 Related Works

Unified World Action Models. Recent world-action models couple video and action generation in a single framework. LingBot-VA [15] casts policy generation as autoregressive video-action diffusion through a shared transformer backbone; Motus [1] adopts a Mixture-of-Transformers architecture coupling a vision-language model, video generator, and action generator via cross-attention; and DreamZero [35] integrates inference-time optimizations that reduce denoising steps at the architecture level. They share an inference-time bottleneck dominated by iterative video and action denoising. A complementary line of work reduces this bottleneck by avoiding test-time video generation: GigaWorld-Policy [34] treats future visual dynamics as a reasoning signal under a causal mask rather than an explicit prediction, and Fast-WAM [38] repurposes a pretrained video DiT as a single-pass encoder for action generation. Our Flash-WAM follows a different direction. Rather than removing video generation, we accelerate it through step distillation, preserving the original WAM inference structure while collapsing each modality’s denoising into one step.

Step Distillation. Recent works compress the iterative denoising process of diffusion models into a small number of inference steps, broadly organized into two families. *Trajectory-following* methods [28, 29, 22, 9, 8] train the student to follow the teacher’s ODE trajectory: progressive distillation [28] iteratively halves the number of sampling steps, and consistency models [29, 22, 9] enforce that any point on the trajectory maps to the same clean endpoint. *Distribution-matching* methods [37, 36, 21, 23, 33, 7] instead train the student so that its output distribution matches the teacher’s, using auxiliary score networks and KL-style or adversarial objectives. Both families have been extended to video diffusion, addressing the additional cost of high-dimensional spatiotemporal tokens [32, 31, 5, 14, 24]. However, these methods are designed for single-modality generation under a single noise distribution. Trajectory-following methods are particularly attractive for our setting because they integrate cleanly into existing flow-matching frameworks and admit the analytical treatment of gradient signal we develop in Section 4.1.

3 Preliminaries

Modern world-action models generate future visual states and the corresponding action sequences using flow matching, a continuous-time generative process that transforms noise into data through iterative denoising. The cost of this iterative process motivates step distillation, in which a *student* model θ_S is trained to reproduce the output of a pretrained *teacher* model θ_T in fewer denoising steps. We review flow matching (Section 3.1) as the underlying generative framework and consistency distillation (Section 3.2) as the acceleration mechanism, then formalize the joint video-action setting considered in this work (Section 3.3).

3.1 Flow Matching

Flow matching [19, 6] is a continuous-time generative framework that learns to transport samples from a noise distribution to a data distribution along straight-line interpolation paths. Clean data $\mathbf{x}_0$ is corrupted to $\mathbf{x}_\sigma = (1 - \sigma) \mathbf{x}_0 + \sigma \epsilon$ with $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and $\sigma \in [0, 1]$. A neural network v_θ is trained to predict the velocity $v = \epsilon - \mathbf{x}_0 = d\mathbf{x}_\sigma/d\sigma$ via the flow matching objective:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\mathbf{x}_0, \epsilon, \sigma} \|v_\theta(\mathbf{x}_\sigma, \sigma) - (\epsilon - \mathbf{x}_0)\|^2, \quad (1)$$

from which the clean estimate is recovered as $\hat{\mathbf{x}}_0 = \mathbf{x}_\sigma - \sigma v_\theta$. To control where training mass concentrates along the noise schedule, an SNR-shifted sampler is commonly used:

$$\sigma = \frac{s \tilde{\sigma}}{1 + (s - 1) \tilde{\sigma}}, \quad \tilde{\sigma} \sim \mathcal{U}[0, 1], \quad (2)$$

parametrized by a shift $s \geq 1$; larger s pushes the distribution toward higher noise levels. Samples are generated at inference by numerical Euler integration of the velocity field from $\sigma = 1$ to $\sigma = 0$.

3.2 Consistency Distillation

Consistency models [29, 22, 9] accelerate sampling by enforcing the *consistency property*: a consistency function $f(\mathbf{x}_\sigma, \sigma)$ maps any point on the probability flow Ordinary Differential Equation (ODE) trajectory to its clean endpoint at $\sigma = 0$. The general form is

$$f(\mathbf{x}_\sigma, \sigma) = a(\sigma) \mathbf{x}_\sigma + b(\sigma) v_\theta(\mathbf{x}_\sigma, \sigma), \quad (3)$$

where $a, b : [0, 1] \rightarrow \mathbb{R}$ satisfy the boundary condition $a(0) = 1$, $b(0) = 0$ that enforces $f(\mathbf{x}_0, 0) = \mathbf{x}_0$. Standard parametrization [12, 29] takes $a(\sigma) = c_{\text{skip}}(\sigma) + c_{\text{out}}(\sigma)$ and $b(\sigma) = -c_{\text{out}}(\sigma) \sigma$, with $c_{\text{skip}} = \sigma_d^2 / (\sigma^2 + \sigma_d^2)$ and $c_{\text{out}} = \sigma \sigma_d / \sqrt{\sigma^2 + \sigma_d^2}$.

In the distillation setting, a frozen teacher θ_T provides guided Euler steps while a student θ_S and an Exponential Moving Average (EMA) target $\theta_{S'}$ are trained to agree along these trajectories. At each iteration, a noise level σ_s is sampled, the schedule is advanced k discrete steps to obtain $\sigma_e < \sigma_s$, and the target $\tilde{\mathbf{x}}_{\sigma_e} = \mathbf{x}_{\sigma_s} + \hat{v}_{\text{cfg}}(\sigma_e - \sigma_s)$ is formed via a teacher Euler step (with classifier-free guidance during distillation). The student is trained against this target via the consistency loss:

$$\mathcal{L}_{\text{CD}} = d(f_{\theta_S}(\mathbf{x}_{\sigma_s}, \sigma_s), f_{\theta_{S'}}(\tilde{\mathbf{x}}_{\sigma_e}, \sigma_e)), \quad (4)$$

where d is a distance metric.

3.3 Problem Formulation

World-action models (WAMs) decompose policy generation into two coupled stages: *visual dynamics prediction*, which predicts how the world will evolve in latent space, and *inverse dynamics*, which recovers the actions consistent with that predicted transition. Given a context $\mathbf{C}$ summarizing past observations, past actions, and a language instruction, a WAM jointly samples a chunk of K future video latents $\mathbf{x}^v$ and the corresponding action sequence $\mathbf{x}^a$:

$$\mathbf{x}^v \sim p_\theta(\mathbf{x}^v \mid \mathbf{C}) \quad (\text{visual dynamics}) \quad (5)$$

$$\mathbf{x}^a \sim p_\theta(\mathbf{x}^a \mid \mathbf{x}^v, \mathbf{C}) \quad (\text{inverse dynamics}) \quad (6)$$

This autoregressive factorization grounds actions in predicted future states. Both stages share the same transformer parameters θ , and each stage is realized as flow matching (Eq. 1): visual dynamics

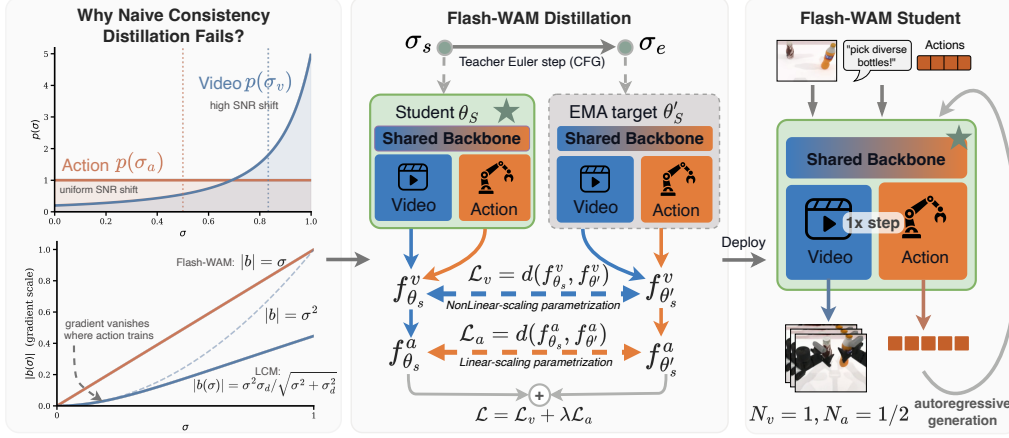


Figure 2: Overview of Flash-WAM. **Left:** the diagnostic motivation showing why naive consistency distillation fails on joint video-action models. **Middle:** the Flash-WAM training pipeline with modality-aware consistency functions. **Right:** the distilled *student* at deployment, autoregressively generating video and actions with single denoising step.

is sampled by Euler integration of a video velocity field v_θ^v over N^v steps, and inverse dynamics by integration of an action velocity field v_θ^a over N^a steps. Generating one chunk therefore requires $N^v + N^a$ sequential transformer forward passes, dominating per-chunk latency and preventing real-time control. We use the shorthand $N^v v / N^a a$ to denote a specific Number of Function Evaluation (NFE) configuration; for example, 25v/50a denotes 25 video and 50 action denoising steps.

Reducing the per-chunk denoising cost via step distillation (Eq. 4) is the natural path to real-time deployment. However, the joint video-action setting departs from the single-modality regime in which distillation methods are typically designed. Following standard practice [15, 35], the two stages use independent SNR-shifted schedulers (Eq. 2) with per-modality shift parameters s^v (video) and s^a (action) satisfying

$$s^v > s^a, \quad (7)$$

reflecting that high-dimensional, structurally redundant video latents tolerate heavier per-step noise, while low-dimensional, precision-critical action sequences require a gentler schedule. Because the two schedulers concentrate training mass at different parts of the noise schedule, the two modalities reach the consistency-distillation loss (Eq. 4) in structurally different noise regimes: a single consistency function $f(\mathbf{x}_\sigma, \sigma) = a(\sigma)\mathbf{x}_\sigma + b(\sigma)v_\theta$ (Eq. 3) applied uniformly across them cannot serve both at once. This is the central obstacle that Flash-WAM addresses, and the focus of Section 4.

4 Methodology

We introduce **Flash-WAM**, a step-distillation framework for joint video-action diffusion models. Under the asymmetric noise schedules of Section 3.3 ($s^v > s^a$), the two streams reach the consistency-distillation loss in structurally different noise regimes, and the consistency function applied to each stream must be selected to match its regime. Section 4.1 establishes why off-the-shelf consistency distillation fails when applied uniformly to both modalities. Section 4.2 then derives Flash-WAM’s modality-aware consistency functions, with parametrizations selected to match each modality’s marginal noise distribution and a joint training objective that distills both streams together.

4.1 The Joint Distillation Regime

The most direct approach to distilling a joint video-action diffusion model is to apply a single consistency function uniformly across both modalities. In the joint regime defined by Section 3.3, this assumption fails since the two modalities reach the loss with substantially different marginal σ distributions. The video stream concentrates near the upper end of $[0, 1]$, while the action stream

spreads across the full range, placing substantial training mass at low σ . We show that this asymmetry produces a structural failure mode rather than a tunable inefficiency, motivating a framework that handles each modality’s regime explicitly.

We trace this failure to the gradient signal that the consistency loss provides at each noise level. Recall from Section 3.2 that any valid consistency function takes the form $f(\mathbf{x}_\sigma, \sigma) = a(\sigma) \mathbf{x}_\sigma + b(\sigma) v_\theta$ with boundary condition $a(0) = 1, b(0) = 0$. Since f depends on θ only through v_θ , the gradient of the consistency loss with respect to θ scales pointwise as $|b(\sigma)|$: whenever $|b(\sigma)|$ is small, the network receives little learning signal at noise level σ regardless of the prediction quality of v_θ . The choice of b therefore determines where in the noise schedule the model can effectively learn.

As a concrete representative of this family, consider the standard LCM parametrization with $b_{\text{LCM}}(\sigma) = -\sigma^2 \sigma_d / \sqrt{\sigma^2 + \sigma_d^2}$. Both $b_{\text{LCM}}(0) = 0$ and $b'_{\text{LCM}}(0) = 0$, so a Taylor expansion at zero gives $|b_{\text{LCM}}(\sigma)| = \sigma^2 / \sigma_d + \mathcal{O}(\sigma^4)$ which is a quadratic vanishing as $\sigma \rightarrow 0$. The left panel of Figure 2 quantifies the gap: at $\sigma = 0.1$, LCM’s gradient-scale factor $|b_{\text{LCM}}(\sigma)|$ (blue line) is roughly $36\times$ smaller than the factor at the high- σ regime where video lives. The quadratic vanishing is not specific to LCM but reflects where in the consistency-function family LCM sits. The following result characterizes the best achievable scaling near $\sigma = 0$:

Proposition 1 (Optimal gradient scaling near $\sigma = 0$). *Let $f(\mathbf{x}_\sigma, \sigma) = a(\sigma) \mathbf{x}_\sigma + b(\sigma) v_\theta$ be any consistency function with $a, b \in C^1([0, 1])$ satisfying $a(0) = 1, b(0) = 0$. Then $|b(\sigma)| = \mathcal{O}(\sigma)$ as $\sigma \rightarrow 0$, and this bound is attained if and only if $b'(0) \neq 0$.*

Proof. By Taylor’s theorem at $\sigma = 0$ with $b(0) = 0$, $b(\sigma) = b'(0)\sigma + \mathcal{O}(\sigma^2)$, so $|b(\sigma)| \leq |b'(0)|\sigma + \mathcal{O}(\sigma^2)$. The leading term vanishes iff $b'(0) = 0$, in which case $|b(\sigma)| = \mathcal{O}(\sigma^2)$. $\square$

LCM falls in the suboptimal case $b'(0) = 0$. The proposition shows that any consistency function with $b'(0) \neq 0$ achieves linear scaling in the low- σ regime, whereas LCM achieves only quadratic scaling. We note that this obstruction is structural rather than parametric. For every choice of σ_d , the inequality $\sigma^2 \sigma_d / \sqrt{\sigma^2 + \sigma_d^2} \leq \sigma$ holds, so no LCM-family member reaches the linear bound of Proposition 1. We therefore look outside the LCM family for the action-stream consistency function.

4.2 Modality-Aware Consistency Functions

Action stream. We now construct consistency functions matched to each modality’s noise regime, beginning with the action stream. The action stream concentrates training mass in the low- σ regime where Proposition 1 is decisive. The simplest pair (a, b) satisfying $a(0) = 1, b(0) = 0$, and $b'(0) \neq 0$ is as follows:

$$a(\sigma) = 1, \quad b(\sigma) = -\sigma, \quad (8)$$

in which b is exactly linear in σ (no higher-order terms to dampen the gradient), a is constant (the consistency target depends on v_θ uniformly across σ rather than being shadowed by a varying skip term), and neither involves a tunable hyperparameter. The resulting consistency function for the action stream is

$$f^a(\mathbf{x}_\sigma^a, \sigma) = 1 \cdot \mathbf{x}_\sigma^a - \sigma \cdot v_\theta(\mathbf{x}_\sigma^a, \sigma). \quad (9)$$

The boundary condition $f^a(\mathbf{x}_0^a, 0) = \mathbf{x}_0^a$ holds by construction, and the consistency property is enforced exactly as in standard consistency distillation [29, 22]. By design, $|b(\sigma)| = \sigma$ throughout $[0, 1]$, achieving the linear scaling of Proposition 1. The derivation here clarifies its role within Flash-WAM as the canonical low- σ realization of the consistency-function family, selected by the framework’s matching principle rather than imported as a parametrization choice.

Video stream. The action-side selection criterion (linear scaling near $\sigma = 0$) does not apply to the video stream. With high s^v , the video distribution concentrates at large σ , where LCM already provides ample gradient signal. In this regime, Flash-WAM’s selection criterion shifts to the high- σ stability properties that the Karras parametrization [12] provides:

- *Variance preservation.* The Karras parametrization keeps $\text{Var}[f] \approx \sigma_d^2$ uniformly in σ , holding the network’s effective input/output ranges stable. Under Eq. (8), $\text{Var}[\mathbf{x}_\sigma - \sigma v_\theta]$ grows with σ , amplifying any prediction error by a factor of σ .

Table 1: Success rates on RoboTwin 2.0 simulation (Clean and Randomized splits, 50 tasks) and speedup over the LingBot-VA as the *teacher*. “*” indicates results we have reproduced.

Method	N^v	N^a	Clean	Rand.	Average	Speedup
π_0 [3]	–	–	65.92	58.40	62.2	–
$\pi_{0.5}$ [11]	–	–	82.74	76.76	79.8	–
X-VLA [40]	–	–	72.9	72.8	72.8	–
Motus [1]	–	–	88.66	87.02	87.8	–
LingBot-VA* [15]	25	50	91.64	90.86	91.25	1.0×
LingBot-VA + DMD2	1	2	85.08	72.36	78.74	19.0×
LingBot-VA + Video-only LCM	1	2	80.66	76.92	78.79	
LingBot-VA + Naive Joint LCM	1	2	25.88	22.07	23.97	
Ours	1	2	88.42	82.66	85.54	
LingBot-VA + DMD2	1	1	52.66	48.46	50.56	23.3×
LingBot-VA + Video-only LCM	1	1	77.90	69.46	73.68	
LingBot-VA + Naive Joint LCM	1	1	39.68	32.96	36.32	
Ours	1	1	82.56	80.26	81.41	

- *Bounded output range.* At high noise, $c_{\text{out}} \rightarrow \sigma_d$ caps the output magnitude, while Eq. (8) has no such bound and can drift outside the data manifold during early training.

For high-dimensional video latents these properties have a direct numerical impact; for low-dimensional, bounded action targets they are largely irrelevant. Flash-WAM therefore selects the LCM parameterization for the video stream:

$$f^v(\mathbf{x}_\sigma^v, \sigma) = c_{\text{skip}}(\sigma) \mathbf{x}_\sigma^v + c_{\text{out}}(\sigma) \hat{\mathbf{x}}_0^v. \quad (10)$$

Joint training objective. The student is trained to satisfy the consistency property in both modalities simultaneously. Each modality contributes a consistency loss using its own consistency function:

$$\mathcal{L}^v = d\left(f_{\theta_S}^v(\mathbf{x}_{\sigma_s}^v, \sigma_s), f_{\theta_{S'}}^v(\tilde{\mathbf{x}}_{\sigma_e}^v, \sigma_e)\right), \quad \mathcal{L}^a = d\left(f_{\theta_S}^a(\mathbf{x}_{\sigma_s}^a, \sigma_s), f_{\theta_{S'}}^a(\tilde{\mathbf{x}}_{\sigma_e}^a, \sigma_e)\right), \quad (11)$$

where the teacher Euler step uses CFG with $w \sim \mathcal{U}[w_{\min}, w_{\max}]$ for video and the unguided prediction for action. The full Flash-WAM objective combines both:

$$\mathcal{L} = \mathcal{L}^v + \lambda_a \mathcal{L}^a. \quad (12)$$

Both consistency targets are computed from a single forward pass per model: video and action tokens are concatenated into the joint sequence used in pre-training and processed by the shared transformer with flex attention. The modality-aware parameterization therefore affect only the per-stream loss heads, leaving the architecture and per-step compute cost unchanged from the teacher.

Flash-WAM’s contribution lies in this principled selection: the per-modality parametrizations are well-known members of the consistency-function family, but the framework explains which to use where, and why.

5 Experiments

5.1 Experimental Setup

We apply our method to the released LingBot-VA model [15] (shared backbone version), a state-of-the-art, open-source world-action model whose parameter count is small enough for commodity edge deployment, where step distillation has the most practical impact. Other recent WAMs (Motus [1], DreamZero [35]) adopt different architectural formulations or integrate their own inference-optimization stacks at the architecture level, and fall outside the scope of our analysis. Per-chunk latency is measured on a single NVIDIA L40S GPU. Although no formal threshold exists for real-time chunked diffusion-based manipulation, we adopt 500 ms (a 2 Hz chunk-level rate) as our real-time budget, consistent with operating points reported in prior work [2, 30] (Figure 1).

Table 2: Success rates on LIBERO benchmarks (Spatial, Object, Goal, Long-horizon) and speedup over the LingBot-VA *teacher*. “*” indicates results we have reproduced.

Method	N^v	N^a	Spatial	Object	Goal	Long	Average	Speedup
π_0 [3]	–	–	96.8	98.8	95.8	85.2	94.1	–
X-VLA [40]	–	–	98.2	98.6	97.8	97.6	98.1	–
LingBot-VA* [15]	20	50	98.5	99.8	98.0	98.3	98.6	1.0×
LingBot-VA + Video-only LCM	1	2	95.1	92.0	96.0	97.8	95.2	13.7×
Ours	1	2	97.0	92.8	96.4	98.0	95.7	13.7×
LingBot-VA + Video-only LCM	1	1	95.0	91.5	95.0	95.4	94.2	16.3×
Ours	1	1	96.0	92.6	96.0	95.8	95.1	16.3×

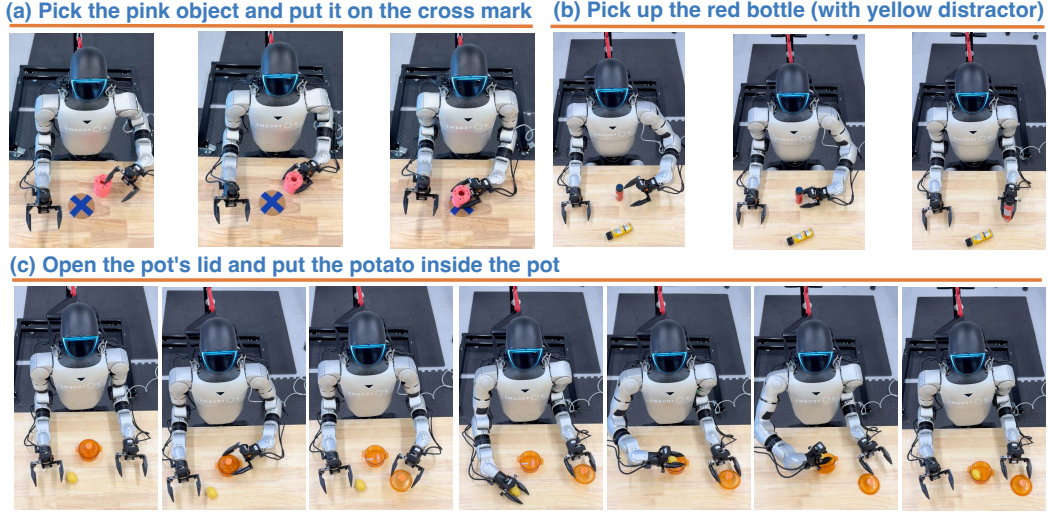


Figure 3: Real-world evaluation suite on Unitree G1 humanoid robot.

Benchmarks. We evaluate on two simulation benchmarks and a real-robot setup. RoboTwin 2.0 [4] is a bimanual manipulation benchmark covering 50 tasks under two evaluation settings: a Clean split with fixed initial configurations, and a Randomized split where object poses, lighting, and scene layouts are perturbed at evaluation time to test robustness to distribution shift. LIBERO [20] comprises four task suites—Spatial, Object, Goal, and Long-horizon—with 500 demonstrations per suite. For real-world evaluation, we deploy on a Unitree G1 humanoid robot equipped with **Unitree Dex1-1 grippers** across three manipulation tasks: (T1) opening a pot’s lid and placing a potato inside, (T2) picking a red bottle from a scene that also contains a yellow distractor bottle, and (T3) picking a pink object and placing it on a marked target location. We collect 50 teleoperated demonstrations per task and report success rates over 10 independent rollouts per task per method.

Baselines. We compare Flash-WAM against off-the-shelf step-distillation algorithms reimplemented for joint video-action generation models. **Naive joint LCM** applies the standard LCM consistency function [22] uniformly across video and action streams, serving as the direct counterpart to our method. **DMD2** adapts Distribution Matching Distillation [36] to LingBot-VA’s video stream, with a flow-matching regularizer on the action stream to stabilize action behavior under the distilled video. **Video-only LCM** distills only the video stream while leaving the action stream unchanged. We further report reference VLA baselines (π_0 , $\pi_{0.5}$, X-VLA, Motus) for context on absolute task performance. Full implementation details for our method and all baselines, including hyperparameters and training configurations, are provided in Appendix A.

5.2 Main Results

RoboTwin. Table 1 reports success rates on RoboTwin 2.0. Flash-WAM at 1v/2a achieves 85.54% average success, recovering most of LingBot-VA’s 91.25% at a 19×

Table 3: Real-world evaluation results on the humanoid Unitree G1.

Method	N^v / N^a	T1	T2	T3	Average
LingBot-VA [15]	3 / 10	50%	70%	80%	66.7%
LingBot-VA (reduced NFE)	1 / 2	30%	30%	60%	40.0%
LingBot-VA + Video-only LCM	1 / 2	30%	50%	50%	43.3%
Flash-WAM	1 / 2	50%	60%	70%	60.0%
LingBot-VA (reduced NFE)	1 / 1	10%	30%	30%	23.3%
LingBot-VA + Video-only LCM	1 / 1	20%	40%	40%	33.3%
Flash-WAM	1 / 1	40%	50%	60%	50.0%

unaccelerated configuration despite reducing video denoising by $25\times$ and action denoising by $50\times$. As shown in Figure 1, the corresponding $23.3\times$ speedup brings per-chunk latency down to 348 ms on a single NVIDIA L40S, enabling real-time inference. Off-the-shelf distillation methods fall well short at the same NFE budgets. At 1v/2a, naive joint LCM collapses to 23.97%, DMD2 reaches 78.74%, and video-only LCM trails at 78.79%. The pattern persists at 1v/1a: naive joint LCM and DMD2 degrade further, while video-only LCM drops to 73.68%. Flash-WAM also surpasses the strongest VLA reference baselines (π_0 , $\pi_{0.5}$, X-VLA) and remains competitive with Motus.

LIBERO. Table 2 reports success rates on LIBERO across its four task suites (Spatial, Object, Goal, Long-horizon). Flash-WAM at 1v/2a achieves 95.7% average success, recovering nearly all of the LingBot-VA teacher’s 98.6% at a $13.7\times$ speedup, and outperforms Video-only LCM on every suite. At 1v/1a, Flash-WAM achieves 95.1% average success at a $16.3\times$ speedup, reducing per-chunk latency from 6,767 ms to 404 ms on NVIDIA L40S and crossing the real-time control budget.

Real-World Experiments. Table 3 reports real-world success rates. The released LingBot-VA model is deployed at 3v/10a and achieves 66.7% average success. Reducing the NFE of LingBot-VA without distillation collapses real-world performance to 40.0% at 1v/2a and 23.3% at 1v/1a, with the pot-and-potato task hit hardest. Applying LCM naively (Video-only LCM) partially recovers performance (43.3% at 1v/2a, 33.3% at 1v/1a), but Flash-WAM substantially outperforms both, reaching 60.0% average at 1v/2a and 50.0% at 1v/1a. The pattern is consistent across all three tasks and both NFE configurations, with the largest absolute gains on the tasks most affected by reduced denoising (T1 at 1v/1a: 10% $\rightarrow$ 40%).

Qualitative analysis. Figure 4 shows representative frames of video predictions from an open-loop autoregressive rollout on a RoboTwin Clean-split task, in which the model generates all subsequent video chunks without intermediate observation feedback. The unaccelerated LingBot-VA teacher (25v/50a) produces clean predictions with object identity and gripper geometry preserved throughout. Both off-the-shelf distillation baselines (naive joint LCM and DMD2) degrade visibly under the same 1v/2a NFE budget as our method: the brown bottle disappears entirely under naive joint LCM and becomes blurred under DMD2. Our method preserves recognizable scene structure and object identity across the rollout. We emphasize that this figure illustrates video-prediction quality only; action precision is captured quantitatively in the success-rate tables.

5.3 Ablation Analysis

Table 4 compares Flash-WAM against three alternative LCM-based distillation strategies on RoboTwin: distilling both modalities uniformly (Naive joint LCM), distilling only the video stream (Video-only LCM), or distilling video while anchoring action behavior with an MSE regularizer (Video-only LCM + reg.). Across both NFE configurations and all task horizons, Flash-WAM outperforms every alternative on Clean and Randomized splits. Naive joint LCM collapses entirely, dropping to 25.88% (Clean split) at 1v/2a with near-zero success at horizons 2 and 3, confirming the analysis of Section 4.1. Video-only LCM avoids this collapse by leaving the action stream at full teacher NFE, but still trails Flash-WAM by roughly 7 points on average at 1v/2a, showing that proper distillation of the action stream is necessary rather than optional. Adding an MSE regularizer recovers 6 points over plain video-only LCM at 1v/2a but degrades at the more aggressive 1v/1a configuration,

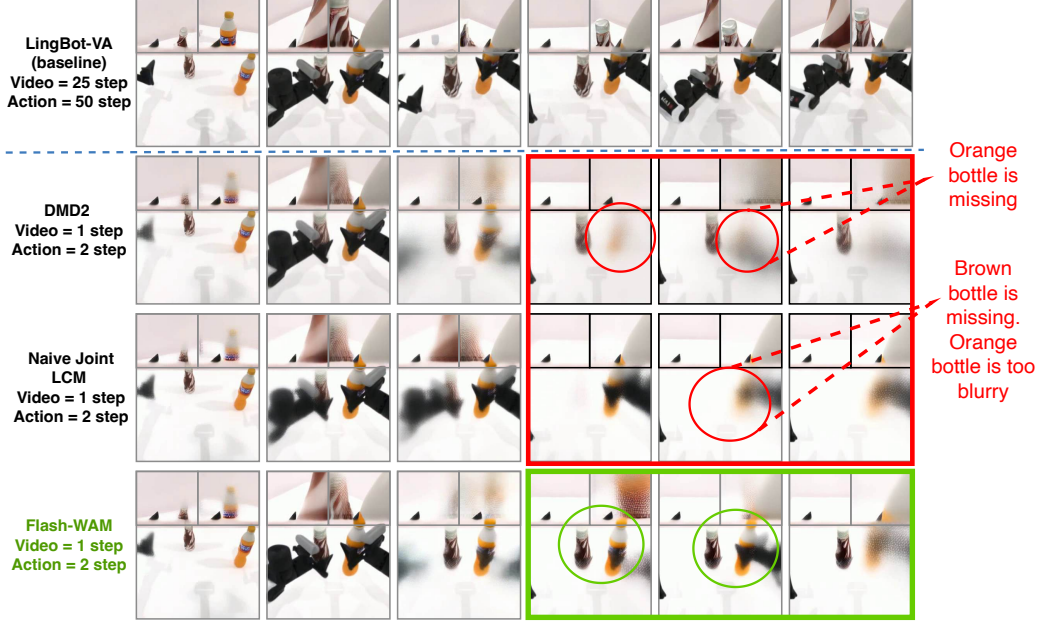


Figure 4: Qualitative comparison on RoboTwin task “pick_diverse_bottles”, generated with open-loop setting.

Table 4: Ablation analysis on RoboTwin 2.0. Comparing four LCM-based distillation strategies against the original LingBot-VA at two NFE configurations, broken down by task horizon (1, 2, 3 sequential steps).

Method	N^v	N^a	Horizon = 1		Horizon = 2		Horizon = 3		Average (50 tasks)	
			Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.
LingBot-VA teacher	25	50	94.18	93.56	90.35	86.95	93.22	93.28	92.93	91.55
Video-only LCM	1	2	87.10	82.73	73.13	68.19	62.50	68.25	80.66	76.92
Video-only LCM + reg.	1	2	<u>91.53</u>	<u>88.50</u>	<u>83.00</u>	<u>74.69</u>	<u>68.00</u>	<u>62.75</u>	<u>86.92</u>	<u>82.02</u>
Naive Joint LCM	1	2	41.00	35.13	4.00	3.13	0.00	0.00	25.88	20.08
Flash-WAM	1	2	92.30	88.47	84.88	76.63	73.50	63.25	88.42	82.66
Video-only LCM	1	1	<u>85.57</u>	<u>78.17</u>	72.06	61.81	43.75	34.75	77.90	69.46
Video-only LCM + reg.	1	1	66.87	61.07	39.19	35.56	10.25	4.75	53.48	48.40
Naive Joint LCM	1	1	54.63	46.00	21.56	15.63	0.00	0.00	39.68	32.96
Flash-WAM	1	1	87.30	86.93	78.44	72.63	63.50	60.75	82.56	80.26

falling 24 points below plain video-only LCM: an auxiliary loss cannot substitute for distilling the action stream when the action NFE budget is tight. Flash-WAM’s modality-aware parametrization is therefore the only strategy that preserves teacher-level accuracy across both NFE configurations and across horizons.

6 Conclusion

We introduced Flash-WAM, a step-distillation framework for joint video-action diffusion models. Our analysis identifies a structural failure mode in off-the-shelf consistency distillation. Asymmetric per-modality noise schedules cause the two streams to reach the distillation loss in different regimes, where a single consistency function cannot serve both. Flash-WAM resolves this by selecting different members of the consistency-function family for each modality, matched to its noise regime. Instantiated on LingBot-VA, the framework recovers near-original task success (85.5% on RoboTwin 2.0 and 95.7% on LIBERO) at $19\times$ speedup, and reaches $23\times$ speedup at a single step with real-time per-chunk latency. Real-world experiments on a Unitree G1 humanoid robot confirm this trend, with Flash-WAM achieving 60% across three manipulation tasks, substantially outperforming both

reduced-NFE inference without distillation (40%) and Video-only LCM (43.3%) at the same step budget.

References

- [1] Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, Chendong Xiang, Yinze Rong, Hongyan Zhao, Hanyu Liu, Zhizhong Su, Lei Ma, Hang Su, and Jun Zhu. Motus: A unified latent action world model, 2025. URL <https://arxiv.org/abs/2512.13030>.
- [2] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies, 2025. URL <https://arxiv.org/abs/2506.07339>.
- [3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2026. URL <https://arxiv.org/abs/2410.24164>.
- [4] Tianxing Chen, Zanzin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [5] Zihan Ding, Chi Jin, Difan Liu, Haitian Zheng, Krishna Kumar Singh, Qiang Zhang, Yan Kang, Zhe Lin, and Yuchen Liu. Dollar: Few-step video generation via distillation and latent reward optimization, 2024. URL <https://arxiv.org/abs/2412.15689>.
- [6] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, Kyle Lacey, Alex Goodwin, Yannik Marek, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis, 2024. URL <https://arxiv.org/abs/2403.03206>.
- [7] Xiangyu Fan, Zesong Qiu, Zhuguanyu Wu, Fanzhou Wang, Zhiqian Lin, Tianxiang Ren, Dahua Lin, Ruihao Gong, and Lei Yang. Phased dmd: Few-step distribution matching distillation via score matching within subintervals, 2026. URL <https://arxiv.org/abs/2510.27684>.
- [8] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models, 2025. URL <https://arxiv.org/abs/2410.12557>.
- [9] Zhengyang Geng, Ashwini Pople, William Luo, Justin Lin, and J. Zico Kolter. Consistency models made easy, 2024. URL <https://arxiv.org/abs/2406.14548>.
- [10] Bohan Hou, Gen Li, Jindou Jia, Tuo An, Xinying Guo, Sicong Leng, Haoran Geng, Yanjie Ze, Tatsuya Harada, Philip Torr, Oier Mees, Marc Pollefeys, Zhuang Liu, Jiajun Wu, Pieter Abbeel, Jitendra Malik, Yilun Du, and Jianfei Yang. World model for robot learning: A comprehensive survey, 2026. URL <https://arxiv.org/abs/2605.00080>.
- [11] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054>.
- [12] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models, 2022. URL <https://arxiv.org/abs/2206.00364>.
- [13] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success, 2025. URL <https://arxiv.org/abs/2502.19645>.

- [14] Jiachen Li, Qian Long, Jian Zheng, Xiaofeng Gao, Robinson Piramuthu, Wenhui Chen, and William Yang Wang. T2v-turbo-v2: Enhancing video generation model post-training through data, reward, and conditional guidance design, 2025. URL <https://arxiv.org/abs/2410.05677>.
- [15] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, Nan Xue, Xing Zhu, Yujun Shen, and Yinghao Xu. Causal world modeling for robot control, 2026. URL <https://arxiv.org/abs/2601.21998>.
- [16] Xinqing Li, Xin He, Le Zhang, Min Wu, Xiaoli Li, and Yun Liu. A comprehensive survey on world models for embodied ai, 2025. URL <https://arxiv.org/abs/2510.16732>.
- [17] Junbang Liang, Pavel Tokmakov, Ruoshi Liu, Sruthi Sudhakar, Paarth Shah, Rares Ambrus, and Carl Vondrick. Video generators are robot policies, 2025. URL <https://arxiv.org/abs/2508.00795>.
- [18] Juyi Lin, Amir Taherin, Arash Akbari, Arman Akbari, Lei Lu, Guangyu Chen, Taskin Padir, Xiaomeng Yang, Weiwei Chen, Yiqian Li, Xue Lin, David Kaeli, Pu Zhao, and Yanzhi Wang. Vote: Vision-language-action optimization with trajectory ensemble voting, 2025. URL <https://arxiv.org/abs/2507.05116>.
- [19] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling, 2023. URL <https://arxiv.org/abs/2210.02747>.
- [20] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning, 2023. URL <https://arxiv.org/abs/2306.03310>.
- [21] Yanzuo Lu, Yuxi Ren, Xin Xia, Shanchuan Lin, Xing Wang, Xuefeng Xiao, Andy J. Ma, Xiaohua Xie, and Jian-Huang Lai. Adversarial distribution matching for diffusion distillation towards efficient image and video synthesis, 2025. URL <https://arxiv.org/abs/2507.18569>.
- [22] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference, 2023. URL <https://arxiv.org/abs/2310.04378>.
- [23] Weijian Luo, Zemin Huang, Zhengyang Geng, J. Zico Kolter, and Guo jun Qi. One-step diffusion distillation through score implicit matching, 2024. URL <https://arxiv.org/abs/2410.16794>.
- [24] Weili Nie, Julius Berner, Nanye Ma, Chao Liu, Saining Xie, and Arash Vahdat. Transition matching distillation for fast video generation, 2026. URL <https://arxiv.org/abs/2601.09881>.
- [25] NVIDIA, :, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llonet, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [26] Amir Rasouli, Yangzheng Wu, Zhiyuan Li, Rui Heng Yang, Xuan Zhao, Charles Eret, and Sajjad Pakdamansavaji. How vlas (really) work in open-world environments, 2026. URL <https://arxiv.org/abs/2604.21192>.
- [27] Timothy Rupprecht, Pu Zhao, Amir Taherin, Arash Akbari, Arman Akbari, Yumei He, Sean Duffy, Juyi Lin, Yixiao Chen, Rahul Chowdhury, Enfu Nan, Yixin Shen, Yifan Cao, Haochen Zeng, Weiwei Chen, Geng Yuan, Jennifer Dy, Sarah Ostadabbas, Silvia Zhang, David Kaeli, Edmund Yeh, and Yanzhi Wang. Human cognition in machines: A unified perspective of world models, 2026. URL <https://arxiv.org/abs/2604.16592>.

- [28] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models, 2022. URL <https://arxiv.org/abs/2202.00512>.
- [29] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models, 2023. URL <https://arxiv.org/abs/2303.01469>.
- [30] Yuteng Sun, Haoran Wang, Ruofei Bai, Zhengguo Li, Jun Li, Meng Yee, Chuah, and Wei Yun Yau. Tidal: Temporally interleaved diffusion and action loop for high-frequency vla control, 2026. URL <https://arxiv.org/abs/2601.14945>.
- [31] Fu-Yun Wang, Zhaoyang Huang, Weikang Bian, Xiaoyu Shi, Keqiang Sun, Guanglu Song, Yu Liu, and Hongsheng Li. Animatelcm: Computation-efficient personalized style video generation without personalized video data, 2024. URL <https://arxiv.org/abs/2402.00769>.
- [32] Xiang Wang, Shiwei Zhang, Han Zhang, Yu Liu, Yingya Zhang, Changxin Gao, and Nong Sang. Videolcm: Video latent consistency model, 2023. URL <https://arxiv.org/abs/2312.09109>.
- [33] Yilun Xu, Weili Nie, and Arash Vahdat. One-step diffusion models with f -divergence distribution matching, 2025. URL <https://arxiv.org/abs/2502.15681>.
- [34] Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Hengtao Li, Jie Li, Jindi Lv, Jingyu Liu, Min Cao, Peng Li, Qiuping Deng, Wenjun Mei, Xiaofeng Wang, Xinze Chen, Xinyu Zhou, Yang Wang, Yifan Chang, Yifan Li, Yukun Zhou, Yun Ye, Zhichao Liu, and Zheng Zhu. Gigaworld-policy: An efficient action-centered world-action model, 2026. URL <https://arxiv.org/abs/2603.17240>.
- [35] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World action models are zero-shot policies, 2026. URL <https://arxiv.org/abs/2602.15922>.
- [36] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Fredo Durand, and William T. Freeman. Improved distribution matching distillation for fast image synthesis, 2024. URL <https://arxiv.org/abs/2405.14867>.
- [37] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Fredo Durand, William T. Freeman, and Taesung Park. One-step diffusion with distribution matching distillation, 2024. URL <https://arxiv.org/abs/2311.18828>.
- [38] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-wam: Do world action models need test-time future imagination?, 2026. URL <https://arxiv.org/abs/2603.16666>.
- [39] Zhanguang Zhang, Zhiyuan Li, Behnam Rahmati, Rui Heng Yang, Yintao Ma, Amir Rasouli, Sajjad Pakdamansavoji, Yangzheng Wu, Lingfeng Zhang, Tongtong Cao, Feng Wen, Xinyu Wang, Xingyue Quan, and Yingxue Zhang. Do world action models generalize better than vlas? a robustness study, 2026. URL <https://arxiv.org/abs/2603.22078>.
- [40] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Ya-Qin Zhang, Jiangmiao Pang, Jingjing Liu, Tai Wang, and Xianyu Zhan. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model, 2025. URL <https://arxiv.org/abs/2510.10274>.
- [41] Yifan Zhong, Xuchuan Huang, Ruochong Li, Ceyao Zhang, Zhang Chen, Tianrui Guan, Fanlian Zeng, Ka Num Lui, Yuyao Ye, Yitao Liang, Yaodong Yang, and Yuanpei Chen. Dexgraspvla: A vision-language-action framework towards general dexterous grasping, 2025. URL <https://arxiv.org/abs/2502.20900>.

A Implementation Details

We provide full implementation details for our method and all baselines. Section A.1 describes the fine-tuning phase in which the released LingBot-VA base checkpoint is adapted to each LIBERO suite prior to distillation. Section A.2 reports the distillation hyperparameters used to train Flash-WAM, which are largely shared between the LIBERO and RoboTwin experiments. Section A.3 then specifies the implementation choices for each baseline, organized by distillation family. All training is performed on NVIDIA H100 GPUs; evaluation and latency profiling are performed on NVIDIA L40S GPUs.

A.1 Libero Finetuning

The released LingBot-VA checkpoint is a base model trained on multi-task data [15]. Following the LingBot-VA protocol, we first fine-tune this base checkpoint separately on each LIBERO suite for 4,000 training steps before applying step distillation. The fine-tuning hyperparameters are listed in Table 5. Fine-tuning on each suite will take about 24 hours on 4 H100s.

Table 5: Hyperparameters used to fine-tune the LingBot-VA base checkpoint on each LIBERO suite.

Hyperparameter	Value
<i>Optimization</i>	
Optimizer	AdamW, $(\beta_1, \beta_2) = (0.9, 0.95)$
Learning rate	1×10^{-5}
Weight decay	0.1
Warmup steps	10 (linear warmup, then constant)
Gradient clipping	2.0
<i>Batching</i>	
Per-device batch size	1
Gradient accumulation steps	30
Effective batch size	120 ($4 \times$ H100)
<i>Training schedule</i>	
Total training steps	4,000

A.2 Flash-WAM Distillation Hyperparameters

Starting from the fine-tuned LingBot-VA teacher, we apply Flash-WAM for 2,000 steps on each LIBERO suite using the hyperparameters in Table 6. Each suite takes approximately 24 hours on $4 \times$ H100 GPUs. The same training procedure is applied to all LCM-based baselines (Naive joint LCM, Video-only LCM, Video-only LCM + reg.) on RoboTwin for fair comparison.

A.3 Baseline Implementations

We describe each baseline’s specific implementation choices, organized by distillation family. All baselines share the training data, base checkpoint, and number of training iterations with our method (Section A.2); they differ only in the distillation objective applied.

A.3.1 Naive Joint LCM

Naive Joint LCM applies the standard LCM consistency function [22] uniformly across video and action streams. The consistency function takes the form $f(\mathbf{x}_\sigma, \sigma) = c_{\text{skip}}(\sigma)\mathbf{x}_\sigma + c_{\text{out}}(\sigma)\hat{\mathbf{x}}_0$ with $c_{\text{skip}} = \sigma_d^2 / (\sigma^2 + \sigma_d^2)$ and $c_{\text{out}} = \sigma\sigma_d / \sqrt{\sigma^2 + \sigma_d^2}$. The consistency loss is computed independently for each modality and combined as $\mathcal{L} = \mathcal{L}^v + \lambda_a \mathcal{L}^a$ with $\lambda_a = 1.0$, identical to Flash-WAM’s joint training objective. The only difference from Flash-WAM is that the action stream uses the same LCM parametrization as video rather than the linear-scaling parametrization.

Table 6: Hyperparameters used to train our distilled student on LIBERO. The same configuration is used across all four LIBERO suites.

Hyperparameter	Value
<i>Architecture</i>	
Image resolution	128×128
Action dimension	30
Actions per video frame	4
Frame chunk size K	4
<i>Flow matching</i>	
Video SNR shift s^v	5.0
Action SNR shift s^a	1.0
<i>Consistency distillation</i>	
Action loss weight λ_a	1.0
Action regularizer weight λ_r	0.2
EMA decay α	0.995
Data scale σ_d	0.5
Loss type	Huber ($c = 0.001$)
CFG range $[w_{\min}, w_{\max}]$	$[2.0, 10.0]$
<i>Optimization</i>	
Optimizer	AdamW, $(\beta_1, \beta_2) = (0.9, 0.999)$
Learning rate	5×10^{-6}
Gradient clipping	2.0
Warmup steps	100
Effective batch size	48 ($4 \times \text{H100}$)

A.3.2 Video-only LCM

Video-only LCM distills only the video stream while leaving the action stream unchanged at full teacher NFE during inference. During training, the consistency loss is computed only on the video stream. The action stream is not modified during distillation; at inference, the distilled student handles the video forward pass at the reduced NFE while the action stream is denoised at the teacher’s full 50-step schedule. All other hyperparameters match the Flash-WAM configuration in Table 6.

A.3.3 Video-only LCM + reg

Video-only LCM + reg extends Video-only LCM by adding a flow-matching regularizer on the action stream during distillation, allowing both streams to operate at reduced NFE at inference time. The video stream is supervised by the standard LCM consistency loss as in Video-only LCM. The action stream is supervised by an MSE flow-matching loss anchored against the demonstration distribution. Specifically, given a clean ground-truth action $\mathbf{x}_0^a$ and a noise level σ sampled from the action stream’s schedule, the action input is constructed as $\mathbf{x}_\sigma^a = (1 - \sigma) \mathbf{x}_0^a + \sigma \epsilon^a$ with $\epsilon^a \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and the regularizer supervises the student’s action velocity prediction against the target velocity:

$$v^* = \frac{\mathbf{x}_\sigma^a - \mathbf{x}_0^a}{\sigma}. \quad (13)$$

The action regularizer takes the form

$$\mathcal{L}_{\text{reg}}^a = \frac{1}{|\mathcal{M}|} \|\mathcal{M} \odot (v_{\theta_S}^a - v^*)\|^2, \quad (14)$$

where $\mathcal{M}$ is the per-channel action validity mask. The full training objective combines the video consistency loss with the action regularizer:

$$\mathcal{L} = \mathcal{L}_{\text{LCM}}^v + \lambda_r \mathcal{L}_{\text{reg}}^a. \quad (15)$$

All other hyperparameters match the Flash-WAM configuration in Table 6.

A.3.4 DMD2 Baseline Implementations

DMD2 [36] was originally proposed for single-modality image and video distillation. Adapting it to the joint video-action diffusion regime requires several architectural and training decisions that the

original method does not prescribe. We describe our adaptation choices in this section to make our two DMD2 baselines reproducible: **Video-only DMD2 + reg** (used in the main results, Table 1) and **Joint DMD2** (the fully-joint variant evaluated in Appendix B).

Throughout, we use the following notation: θ_T for the unaccelerated LingBot-VA model, θ_S for the student, and θ_C for the critic. We denote the student’s final output as $G_{\theta_S}(z, y) = \hat{\mathbf{x}}_0$, where z is the random seed and y is the conditioning context.

Networks. We maintain three full-copy networks of the joint video-action backbone:

- *Reference model* θ_T (frozen): the pretrained LingBot-VA model. Defines the real score.
- *Student* θ_S (trainable): a K -step generator initialized from θ_T .
- *Critic* θ_C (trainable): tracks the student’s joint generation distribution. Defines the fake score.

A single critic scores both modalities through the shared backbone with separate output heads (the same head structure as the reference model).

Variant overview. The two DMD2 baselines differ in two dimensions: which modalities are generated from noise during student rollouts, and which losses contribute to the student objective. Joint DMD2 (appendix variant): The student generates both video and action from pure noise via K -step denoising. Distribution-matching losses are applied to both modalities. There is no action regularizer. Video-only DMD2 + reg (main paper variant): The student generates only the video stream from pure noise. The action stream input is constructed by perturbing the ground-truth action at each noise level rather than being denoised from noise. The student is supervised by distribution-matching on the video stream and an MSE-based action regularizer on the action stream.

The networks, scoring procedure, and critic objective are shared between the two variants. The variant-specific differences are flagged where they apply in the descriptions below.

Student rollout. The student denoises in K steps (we use $K = 4$) following a uniform noise-band schedule $1 = \sigma_0 > \sigma_1 > \dots > \sigma_K = 0$. At each step i , the student computes the velocity prediction

$$v_{\theta_S}(\mathbf{x}_{\sigma_i}^v, \mathbf{x}_{\sigma_i}^a, \tilde{\mathbf{x}}_0^v, \tilde{\mathbf{x}}_0^a, \sigma_i, y), \quad (16)$$

from which the predicted clean output for video is recovered as $\hat{\mathbf{x}}_0^{v,(i)} = \mathbf{x}_{\sigma_i}^v - \sigma_i v_{\theta_S}^v$. Clean-context tokens $(\tilde{\mathbf{x}}_0^v, \tilde{\mathbf{x}}_0^a)$ supply past-frame context through flex attention, exactly as in pretraining (block-causal across chunks; strict causality from noisy to clean tokens). The video input at the next step is constructed by re-noising the predicted clean video:

$$\mathbf{x}_{\sigma_{i+1}}^v = (1 - \sigma_{i+1}) \hat{\mathbf{x}}_0^{v,(i)} + \sigma_{i+1} \epsilon^{v,(i+1)}, \quad \epsilon^{v,(i+1)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (17)$$

The action stream input depends on the variant. For Joint DMD2, the action stream is denoised symmetrically with the video stream:

$$\mathbf{x}_{\sigma_{i+1}}^a = (1 - \sigma_{i+1}) \hat{\mathbf{x}}_0^{a,(i)} + \sigma_{i+1} \epsilon^{a,(i+1)}, \quad (18)$$

where $\hat{\mathbf{x}}_0^{a,(i)} = \mathbf{x}_{\sigma_i}^a - \sigma_i v_{\theta_S}^a$. For Video-only DMD2 + reg, the action stream input at each step is constructed by directly perturbing the ground-truth action at the corresponding noise level:

$$\mathbf{x}_{\sigma_i}^a = (1 - \sigma_i) \mathbf{x}_0^a + \sigma_i \epsilon^{a,i}, \quad \epsilon^{a,i} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (19)$$

with $\mathbf{x}_0^a$ the ground-truth action. To bound activation memory, only the final step ($i = K - 1$) retains autograd; steps $0, \dots, K - 2$ run under `no_grad`.

Single-pass joint scoring. Given a student rollout output, both modalities are re-noised at independently sampled noise levels $\sigma^v, \sigma^a \sim \mathcal{U}(0.02, 0.98)$ and presented jointly to both the critic and the reference model:

$$\tilde{\mathbf{x}}^v = (1 - \sigma^v) \hat{\mathbf{x}}^v + \sigma^v \boldsymbol{\eta}^v. \quad (20)$$

For Joint DMD2, the action input to scoring is constructed analogously by re-noising the student’s predicted action $\hat{\mathbf{x}}_0^a$. For Video-only DMD2 + reg, the action input to scoring is constructed by

perturbing the ground-truth action $\mathbf{x}_0^a$ at noise level σ^a directly. In both variants, the critic and reference model produce their predictions in a single forward pass on the joint input $(\tilde{\mathbf{x}}^v, \tilde{\mathbf{x}}^a)$.

The fake and real predicted clean outputs for video are

$$\hat{\mathbf{x}}_0^{v,\text{fake}} = \tilde{\mathbf{x}}^v - \sigma^v v_{\theta_C}^v(\tilde{\mathbf{x}}^v, \tilde{\mathbf{x}}^a, \sigma^v, \sigma^a, y), \quad (21)$$

$$\hat{\mathbf{x}}_0^{v,\text{real}} = \tilde{\mathbf{x}}^v - \sigma^v v_{\theta_T}^{v,\text{cfg}}(\tilde{\mathbf{x}}^v, \tilde{\mathbf{x}}^a, \sigma^v, \sigma^a, y), \quad (22)$$

with classifier-free guidance applied only to the video real score:

$$v_{\theta_T}^{v,\text{cfg}} = v_{\theta_T}^v(\cdot | \emptyset) + w^v [v_{\theta_T}^v(\cdot | y) - v_{\theta_T}^v(\cdot | \emptyset)], \quad w^v = 3.0. \quad (23)$$

For Joint DMD2, the analogous fake and real action outputs are computed from the critic and reference model’s action heads (without CFG): $\hat{\mathbf{x}}_0^{a,\text{real}} = \tilde{\mathbf{x}}^a - \sigma^a v_{\theta_T}^a(\cdot | y)$. Total scoring cost per joint sample is three forward passes (one critic, two reference: conditioned and unconditioned).

Distribution-matching losses. The DMD2 distribution-matching gradient pushes the student’s output distribution toward the reference distribution. For video,

$$\mathcal{L}_{\text{DM}}^v = \frac{1}{2} \|G_{\theta_S}^v(z, y) - \text{sg}[G_{\theta_S}^v(z, y) - g^v]\|^2, \quad (24)$$

where the gradient surrogate is

$$g^v = \frac{\hat{\mathbf{x}}_0^{v,\text{fake}} - \hat{\mathbf{x}}_0^{v,\text{real}}}{\|\hat{\mathbf{x}}_0^v - \hat{\mathbf{x}}_0^{v,\text{real}}\|_1 + \varepsilon}, \quad (25)$$

with $\varepsilon = 10^{-8}$ and per-sample L_1 normalization following the DMD2 gradient-norm fix. For Joint DMD2, an analogous distribution-matching loss is applied to the action stream:

$$g^a = \frac{\hat{\mathbf{x}}_0^{a,\text{fake}} - \hat{\mathbf{x}}_0^{a,\text{real}}}{\|\hat{\mathbf{x}}_0^a - \hat{\mathbf{x}}_0^{a,\text{real}}\|_1 + \varepsilon}, \quad (26)$$

$$\mathcal{L}_{\text{DM}}^a = \frac{1}{|\mathcal{M}|} \|\mathcal{M} \odot (G_{\theta_S}^a - \text{sg}[G_{\theta_S}^a - g^a])\|^2, \quad (27)$$

where $\mathcal{M}$ is the per-channel action validity mask. For Video-only DMD2 + reg, no distribution-matching loss is applied to the action stream.

Action regularizer (Video-only DMD2 + reg only). For the Video-only DMD2 + reg variant, we anchor the student’s action head to the demonstration distribution with a flow-matching loss evaluated on the action input at the student’s final generation step. Let $\mathbf{x}_{\sigma_{K-1}}^a$ denote the action input at step $K - 1$ (the perturbed ground-truth action). The target velocity that maps this input to the clean ground-truth action is

$$v^* = \frac{\mathbf{x}_{\sigma_{K-1}}^a - \mathbf{x}_0^a}{\sigma_{K-1}}. \quad (28)$$

We supervise the student’s final-step action prediction against v^* :

$$\mathcal{L}_{\text{reg}}^a = \frac{1}{|\mathcal{M}|} \|\mathcal{M} \odot (v_{\theta_S}^a - v^*)\|^2. \quad (29)$$

Critic objective. The critic is trained to denoise the student’s joint distribution via flow matching. For each rollout, fresh noise levels $\sigma^{v'}$, $\sigma^{a'}$ and noise $\boldsymbol{\eta}^{v'}$, $\boldsymbol{\eta}^{a'}$ are drawn, the student’s joint output is re-noised, and the critic minimizes

$$\mathcal{L}_{\text{critic}} = w(\sigma^{v'}) \|v_{\theta_C}^v - (\boldsymbol{\eta}^{v'} - \hat{\mathbf{x}}_0^v)\|^2 + w(\sigma^{a'}) \|\mathcal{M} \odot [v_{\theta_C}^a - (\boldsymbol{\eta}^{a'} - \hat{\mathbf{x}}_0^a)]\|^2, \quad (30)$$

where $w(\sigma) = \exp(-2((\sigma - 0.5)/T)^2)$ is a bell-curve timestep weight inherited from pretraining. The critic and student updates share a single forward pass through the joint input.

The two DMD2 baselines combine the loss terms above as follows.

Joint DMD2:

$$\mathcal{L}_{\theta_S}^{\text{joint}} = \lambda_{\text{DM}}^v \mathcal{L}_{\text{DM}}^v + \lambda_{\text{DM}}^a \mathcal{L}_{\text{DM}}^a. \quad (31)$$

Video-only DMD2 + reg:

$$\mathcal{L}_{\theta_S}^{\text{V-only}} = \lambda_{\text{DM}}^v \mathcal{L}_{\text{DM}}^v + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}^a. \quad (32)$$

We use $\lambda_{\text{DM}}^v = 1.0$, $\lambda_{\text{DM}}^a = 0.1$, $\lambda_{\text{reg}} = 1.0$. Following DMD2’s update schedule, the critic updates every iteration while the student updates every $T_g = 5$ iterations.

Table 7: Ablation analysis on RoboTwin 2.0. We compare four LCM-based distillation strategies against the unaccelerated LingBot-VA teacher at two NFE configurations, broken down by task horizon (1, 2, 3 sequential steps) and averaged across all 50 tasks.

Method	N^v	N^a	Horizon = 1		Horizon = 2		Horizon = 3		Average (50 tasks)	
			Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.
Joint DMD2	1	1	67.50	62.13	36.50	33.81	6.00	4.50	52.66	48.46
DMD2 + reg.	1	1	76.93	72.23	54.81	50.25	25.00	11.25	66.53	60.32
Video-only LCM	1	1	85.57	78.17	72.06	61.81	43.75	34.75	77.90	69.46
Video-only LCM + reg.	1	1	66.87	61.07	39.19	35.56	10.25	4.75	53.48	48.40
Naive joint LCM	1	1	54.63	46.00	21.56	15.63	0.00	0.00	39.68	32.96
Flash-WAM	1	1	87.30	86.93	78.44	72.63	63.50	60.75	82.56	80.26

Hyperparameters. We use AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, no weight decay, gradient clipping $\|g\|_2 \leq 2.0$, and a 100-step linear warmup to constant learning rate. The student learning rate is 5×10^{-7} and the critic learning rate is 10^{-6} . Both DMD2 variants are trained on $4 \times$ NVIDIA H100 GPUs for 2,000 steps, matching the LCM-based training schedule.

B Additional Experimental Results

Table 7 reports a comprehensive comparison of all distillation strategies at the most aggressive single-step configuration (1v/1a) on RoboTwin 2.0. The table includes Joint DMD2 (Section A.3.4), the fully-joint DMD2 variant excluded from the main paper. Joint DMD2 reaches 52.7% Clean and 48.5% Randomized on average, falling roughly 14 points below the Video-only DMD2 + reg variant reported in the main results and 30 points below Flash-WAM. The pattern is consistent with the diagnosis of Section 4.1: like LCM, naively applying DMD2 uniformly across both modalities cannot serve the asymmetric noise regimes that joint video-action models impose. Restricting DMD2 to the video stream and anchoring action behavior with a regularizer (the Video-only DMD2 + reg variant) substantially improves performance over the joint variant, but still trails Flash-WAM by a wide margin.

C Limitations and future work

Our experiments are in simulation; real-world deployment on physical robots remains for future work. Flash-WAM targets the shared-backbone WAM regime, and extending the framework to multi-model architectures with separate per-modality sub-models is a natural next step. We characterize optimal gradient scaling in the low- σ regime where actions train; a corresponding analysis for the high- σ regime would complete the analytical picture. Finally, the modality-aware selection principle may transfer to distribution-matching distillation methods and to other multi-modal diffusion settings with heterogeneous noise schedules which need further analysis.

D LLM Usage

Large language models were used in a limited and clearly bounded role during the preparation of this paper. Specifically, we used LLMs to assist with writing tasks: improving grammar, rephrasing sentences for clarity, suggesting alternative wordings, and helping to tighten verbose passages. LLMs were also occasionally used to verify that technical phrasings followed standard conventions in the diffusion and step-distillation literature. All technical claims, numerical results, and conclusions in the paper reflect the authors’ own findings and are not generated, suggested, or substantively shaped by LLM output.

Table 8: Per-task success rate results on Robotwin 2.0.

Simulation Task	Horizon	Flash-WAM (1v/2a)		Flash-WAM(1v/1a)		Naive Joint LCM(1v/1a)		DMD2 (1v/1a)	
		Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.
Adjust Bottle	1	98	98	98	98	93	88	98	85
Beat Block Hammer	1	98	95	99	97	61	20	77	74
Blocks Ranking RGB	3	86	85	73	74	0	0	8	5
Blocks Ranking Size	3	68	46	72	65	0	0	5	1
Click Alarmclock	1	100	100	100	100	81	86	100	100
Click Bell	1	100	100	100	100	100	100	100	99
Dump Bin Bigbin	1	95	98	94	93	71	61	67	57
Grab Roller	1	100	100	100	100	98	94	97	97
Handover Block	2	91	51	71	34	0	0	22	11
Handover Mic	2	69	71	66	63	44	25	11	22
Hanging Mug	2	36	39	32	28	5	2	3	4
Lift Pot	1	99	100	98	95	2	8	25	11
Move Can Pot	1	91	80	95	92	2	3	33	16
Move Pillbottle Pad	1	99	94	93	89	25	17	58	58
Move Playingcard Away	1	100	96	100	100	86	82	97	90
Move Stapler Pad	1	61	46	39	36	14	15	16	10
Open Laptop	1	94	87	95	94	74	70	64	62
Open Microwave	1	67	70	25	27	26	27	59	62
Pick Diverse Bottles	2	92	66	92	88	11	4	60	42
Pick Dual Bottles	2	100	85	99	84	4	3	74	56
Place A2B Left	1	91	81	88	93	63	49	72	57
Place A2B Right	1	92	91	91	92	61	46	68	67
Place Bread Basket	1	93	85	89	72	51	31	55	46
Place Bread Skillet	2	89	85	86	88	53	44	51	46
Place Burger Fries	2	97	95	94	93	74	63	80	81
Place Can Basket	2	83	75	79	76	16	15	18	25
Place Cans Plasticbox	2	100	97	96	97	2	1	5	4
Place Container Plate	1	99	98	97	97	87	67	88	94
Place Dual Shoes	2	78	81	65	64	0	2	20	21
Place Empty Cup	1	100	98	99	99	26	27	69	67
Place Fan	1	83	77	65	78	25	20	21	34
Place Mouse Pad	1	89	84	85	80	33	18	48	46
Place Object Basket	2	87	77	78	84	17	14	59	47
Place Object Scale	1	95	86	96	97	47	26	66	52
Place Object Stand	1	100	95	95	92	15	9	69	72
Place Phone Stand	1	97	94	96	95	60	45	87	74
Place Shoe	1	91	92	56	79	32	23	55	46
Press Stapler	1	93	93	89	93	88	78	84	78
Put Bottles Dustbin	3	44	30	19	15	0	0	0	0
Put Object Cabinet	2	79	54	62	39	3	1	6	2
Rotate QRcode	1	94	92	88	82	51	60	56	48
Scan Object	2	88	80	78	68	25	22	29	22
Shake Bottle Horizontally	1	99	95	99	95	97	88	99	94
Shake Bottle	1	99	97	100	97	94	85	99	95
Stack Blocks Three	3	96	92	90	89	0	0	11	12
Stack Blocks Two	2	100	99	100	99	10	4	67	57
Stack Bowls Three	3	76	78	60	67	24	10	19	35
Stack Bowls Two	2	93	93	97	90	57	40	60	66
Stamp Seal	1	96	83	89	85	14	6	31	30
Turn Switch	1	56	49	61	61	62	49	67	43
Average (%)	–	88.42	82.66	82.56	80.26	39.68	32.96	52.66	48.46